Oui.
Je te fais une progression complète, de la manipulation la plus simple jusqu'à une chaîne "synthwave-ish" cohérente avec le brief. Je pars sur Burn **0.20.1** avec un backend CPU `burn-ndarray` pour garder le code lisible, plus `hound` pour lire/écrire des WAV. Burn expose bien les opérations dont on a besoin ici (`sin`, `cos`, `tanh`, `clamp_*`, slicing via `s!`), et son backend `buOui.`

Je te fais une progression complète, de la manipulation la plus simple jusqu'à une chaîne "synthwave-ish" cohérente avec le brief. Je pars sur Burn **0.20.1** avec un backend CPU burn-ndarray pour garder le code lisible, plus hound pour lire/écrire des WAV. Burn expose bien les opérations dont on a besoin ici (sin, cos, tanh, clamp_*, slicing via s!), et son backend burn-wgpu s'appuie sur un CubeBackend<WgpuRuntime, ...> pour exécuter les tenseurs sur GPU. (Docs.rs)

Je te donne une version cohérente avec Burn 0.20.1 ; selon ton setup exact, il peut y avoir 1 ou 2 imports à ajuster.

Cargo.toml

```
[package]

name = "burn-audio-playground"

version = "0.1.0"

edition = "2021"



[dependencies]

burn = "0.20.1"

burn-ndarray = "0.20.1"

hound = "3.5.1"
```

src/main.rs

```
use burn::{

    prelude::s,

    tensor::{Int, Tensor, TensorData},
```

```
};

use burn_ndarray::{NdArray, NdArrayDevice};

use std::{error::Error, f32::consts::PI, path::Path};


///
---------------------------------------------------------------------
-------

/// BACKEND

///
---------------------------------------------------------------------
-------

/// Version simple: CPU avec ndarray.

///

/// Pour passer sur GPU plus tard, remplace simplement ces lignes
par:

///

/// use burn_wgpu::{Wgpu, WgpuDevice};

/// type B = Wgpu;

/// type Device = WgpuDevice;

///

/// et dans main():

/// let device = WgpuDevice::DefaultDevice;

///

/// Le reste du code DSP ne change pas.

///
---------------------------------------------------------------------
-------
```

```
type B = NdArray<f32>;

type Device = NdArrayDevice;


///
---------------------------------------------------------------------
-------

/// I/O WAV (mono 16-bit pour garder le code court et clair)

///
---------------------------------------------------------------------
-------


/// Lit un WAV mono 16-bit et renvoie les samples normalisés en
[-1.0, 1.0].

fn read_wav_mono_f32(path: &str) -> Result<(Vec<f32>,
hound::WavSpec), Box<dyn Error>> {

    let mut reader = hound::WavReader::open(path)?;

    let spec = reader.spec();


    if spec.channels != 1 {

        return Err("Ce code de démo attend un WAV mono.".into());

    }

    if spec.bits_per_sample != 16 || spec.sample_format !=
hound::SampleFormat::Int {

        return Err("Ce code de démo attend un WAV 16-bit
PCM.".into());

    }


    let samples: Vec<f32> = reader
```

```
        .samples::<i16>()
        .map(|s| s.map(|v| v as f32 / i16::MAX as f32))
        .collect::<Result<Vec<_>, _>>()?;

    Ok((samples, spec))
}

/// Écrit un WAV mono 16-bit à partir de samples en [-1.0, 1.0].
fn write_wav_mono_f32(path: &str, samples: &[f32], sample_rate: u32)
-> Result<(), Box<dyn Error>> {
    let spec = hound::WavSpec {
        channels: 1,
        sample_rate,
        bits_per_sample: 16,
        sample_format: hound::SampleFormat::Int,
    };

    let mut writer = hound::WavWriter::create(path, spec)?;
    for &x in samples {
        let y = x.clamp(-1.0, 1.0);
        writer.write_sample((y * i16::MAX as f32) as i16)?;
    }
    writer.finalize()?;
    Ok(())
}
```

```
/// Convertit un Vec<f32> vers un Tensor 1D Burn.

fn tensor_from_samples(samples: &[f32], device: &Device) ->
Tensor<B, 1> {

    Tensor::<B, 1>::from_data(TensorData::new(samples.to_vec(),
[samples.len()]), device)

}



/// Convertit un Tensor Burn vers Vec<f32>.

fn tensor_to_samples(x: Tensor<B, 1>) -> Vec<f32> {

    x.into_data().to_vec::<f32>().unwrap()

}



///
---------------------------------------------------------------------
-------

/// OUTILS DE BASE

///
---------------------------------------------------------------------
-------



/// Crée un axe temporel t = [0, 1/sr, 2/sr, ...].

fn time_axis(n: usize, sample_rate: usize, device: &Device) ->
Tensor<B, 1> {

    Tensor::<B, 1, Int>::arange(0..n as i64, device)

        .float()

        .div_scalar(sample_rate as f32)
```

```
}


/// Génère une onde sinusoïdale pure.
fn sine_wave(freq_hz: f32, seconds: f32, sample_rate: usize, device:
&Device) -> Tensor<B, 1> {
    let n = (seconds * sample_rate as f32) as usize;
    let t = time_axis(n, sample_rate, device);


    // sin(2πft)
    t.mul_scalar(2.0 * PI * freq_hz).sin()
}


/// Normalise un signal pour que son pic soit égal à `target_peak`.
fn normalize_peak(x: Tensor<B, 1>, target_peak: f32) -> Tensor<B, 1>
{
    let peak = x.clone().abs().max().into_scalar();


    // Évite une division par zéro sur un signal silencieux.
    if peak < 1e-8 {
        x
    } else {
        x.div_scalar(peak).mul_scalar(target_peak)
    }
}
```

```
/// Mélange dry/wet.

/// wet_amount = 0.0 => 100% dry

/// wet_amount = 1.0 => 100% wet

fn mix(dry: Tensor<B, 1>, wet: Tensor<B, 1>, wet_amount: f32) ->
Tensor<B, 1> {

    dry.clone().mul_scalar(1.0 - wet_amount) +
wet.mul_scalar(wet_amount)

}



/// Crée un signal de test plus intéressant qu'une simple sinusoïde:

/// un mélange de grave / médium / aigu.

fn demo_source(sample_rate: usize, device: &Device) -> Tensor<B, 1>
{

    let bass = sine_wave(110.0, 4.0, sample_rate,
device).mul_scalar(0.55);

    let mid = sine_wave(220.0, 4.0, sample_rate,
device).mul_scalar(0.30);

    let high = sine_wave(880.0, 4.0, sample_rate,
device).mul_scalar(0.15);



    normalize_peak(bass + mid + high, 0.90)

}



/// Décale le signal vers la droite avec remplissage par zéros.

/// Utile pour construire delays et FIR "maison".

fn shift_right_zero(x: Tensor<B, 1>, shift: usize) -> Tensor<B, 1> {

    let n = x.dims()[0];
```

```
    let device = x.device();

    if shift == 0 {
        return x;
    }
    if shift >= n {
        return Tensor::<B, 1>::zeros([n], &device);
    }

    let core = x.slice(s![..(n - shift)]);
    let mut out = Tensor::<B, 1>::zeros([n], &device);
    out = out.slice_assign(s![(shift)..], core);
    out
}

/// Décale le signal vers la gauche avec remplissage par zéros.
fn shift_left_zero(x: Tensor<B, 1>, shift: usize) -> Tensor<B, 1> {
    let n = x.dims()[0];
    let device = x.device();

    if shift == 0 {
        return x;
    }
    if shift >= n {
```

```
        return Tensor::<B, 1>::zeros([n], &device);
    }

    let core = x.slice(s![(shift)..]);
    let mut out = Tensor::<B, 1>::zeros([n], &device);
    out = out.slice_assign(s![..(n - shift)], core);
    out
}

///
---------------------------------------------------------------------------
/// EXEMPLE 1 — GAIN (le plus simple)
///
---------------------------------------------------------------------------
/// Multiplie chaque sample par un coefficient.
///
---------------------------------------------------------------------------
fn gain(x: Tensor<B, 1>, amount: f32) -> Tensor<B, 1> {
    x.mul_scalar(amount)
}

///
---------------------------------------------------------------------------
/// EXEMPLE 2 — HARD CLIP
```

```
///
-----------------------------------------------------------------------
-------

/// Coupe brutalement les crêtes au-delà d’un seuil.

/// Son plus agressif, plus “sale”.

///
-----------------------------------------------------------------------
-------

fn hard_clip(x: Tensor<B, 1>, threshold: f32) -> Tensor<B, 1> {

    x.clamp_min(-threshold)

        .clamp_max(threshold)

        .div_scalar(threshold)

}



///
-----------------------------------------------------------------------
-------

/// EXEMPLE 3 — SOFT CLIP / SATURATION

///
-----------------------------------------------------------------------
-------

/// tanh() donne une saturation plus douce que le hard-clip.

/// Très pratique pour “redonner du caractère”.

///
-----------------------------------------------------------------------
-------

fn soft_clip_tanh(x: Tensor<B, 1>, drive: f32) -> Tensor<B, 1> {

    x.mul_scalar(drive).tanh()

}
```

```
///
---------------------------------------------------------------------
-------

/// EXEMPLE 4 — TREMOLO

///
---------------------------------------------------------------------
-------

/// Modulation d’amplitude par un LFO lent.

/// C’est un grand classique rétro.

///
---------------------------------------------------------------------
-------

fn tremolo(x: Tensor<B, 1>, sample_rate: usize, lfo_hz: f32, depth:
f32) -> Tensor<B, 1> {

    let n = x.dims()[0];

    let device = x.device();


    let t = time_axis(n, sample_rate, &device);


    // LFO entre 0 et 1.

    let lfo = t

        .mul_scalar(2.0 * PI * lfo_hz)

        .sin()

        .add_scalar(1.0)

        .div_scalar(2.0);
```

```
    // depth = 0 => aucun effet
    // depth = 1 => amplitude suit totalement le LFO
    let amp = lfo.mul_scalar(depth).add_scalar(1.0 - depth);

    x * amp
}

///
---------------------------------------------------------------------
-------
/// EXEMPLE 5 — RING MODULATION
///
---------------------------------------------------------------------
-------
/// On multiplie le signal par une autre sinusoïde.
/// Ça crée des sidebands métalliques / futuristes.
///
---------------------------------------------------------------------
-------
fn ring_mod(x: Tensor<B, 1>, sample_rate: usize, mod_hz: f32) ->
Tensor<B, 1> {
    let n = x.dims()[0];
    let device = x.device();

    let t = time_axis(n, sample_rate, &device);
    let carrier = t.mul_scalar(2.0 * PI * mod_hz).sin();
```

```
    x * carrier
}


///
---------------------------------------------------------------------
-------
/// EXEMPLE 6 — LOW-PASS “MAISON”
///
---------------------------------------------------------------------
-------
/// Ici on fait un FIR très simple: moyenne glissante symétrique.
/// Plus `radius` est grand, plus on lisse les aigus.
///
---------------------------------------------------------------------
-------
fn low_pass_moving_average(x: Tensor<B, 1>, radius: usize) ->
Tensor<B, 1> {
    let n = x.dims()[0];
    let device = x.device();


    let mut acc = Tensor::<B, 1>::zeros([n], &device);
    let mut taps = 0usize;


    // Centre
    acc = acc + x.clone();
    taps += 1;
```

```
    // Voisins gauche / droite
    for k in 1..=radius {
        acc = acc + shift_right_zero(x.clone(), k);
        acc = acc + shift_left_zero(x.clone(), k);
        taps += 2;
    }

    acc.div_scalar(taps as f32)
}

///
---------------------------------------------------------------------
-------
/// EXEMPLE 7 — HIGH-PASS par soustraction
///
---------------------------------------------------------------------
-------
/// On enlève la version low-pass au signal original.
/// Ce qui reste = surtout les variations rapides => le haut du
spectre.
///
---------------------------------------------------------------------
-------
fn high_pass_from_lowpass(x: Tensor<B, 1>, radius: usize) ->
Tensor<B, 1> {
    let low = low_pass_moving_average(x.clone(), radius);
    x - low
}
```

```
///
---------------------------------------------------------------------
-------

/// EXEMPLE 8 — CHORUS TRÈS SIMPLE

///
---------------------------------------------------------------------
-------

/// Un vrai chorus module le délai dans le temps.

/// Ici, on fait une version simple avec deux micro-delays fixes.

/// Ça épaissit déjà bien le son.

///
---------------------------------------------------------------------
-------

fn chorus(x: Tensor<B, 1>, sample_rate: usize) -> Tensor<B, 1> {

    let d1 = (0.015 * sample_rate as f32) as usize; // 15 ms

    let d2 = (0.023 * sample_rate as f32) as usize; // 23 ms


    let wet =

        shift_right_zero(x.clone(), d1).mul_scalar(0.35) +

        shift_right_zero(x.clone(), d2).mul_scalar(0.25);


    normalize_peak(mix(x, wet, 0.40), 0.95)

}


///
---------------------------------------------------------------------
-------
```

```
/// EXEMPLE 9 — MULTI-TAP DELAY
///
---------------------------------------------------------------------
-------
/// Pas de feedback récursif ici: on fabrique plusieurs copies
retardées,
/// de plus en plus faibles. Très simple, mais super utile.
///
---------------------------------------------------------------------
-------
fn multi_tap_delay(
    x: Tensor<B, 1>,
    sample_rate: usize,
    delay_ms: usize,
    taps: usize,
    decay: f32,
    wet_amount: f32,
) -> Tensor<B, 1> {
    let n = x.dims()[0];
    let device = x.device();
    let base_delay = sample_rate * delay_ms / 1000;

    let mut wet = Tensor::<B, 1>::zeros([n], &device);

    for tap in 1..=taps {
        let shift = base_delay * tap;
```

```
        let gain = decay.powi(tap as i32);


        wet = wet + shift_right_zero(x.clone(),
shift).mul_scalar(gain);

    }


    normalize_peak(mix(x, wet, wet_amount), 0.95)

}


///
---------------------------------------------------------------------
-------

/// EXEMPLE 10 — “PUMP” type sidechain

///
---------------------------------------------------------------------
-------

/// Pour le côté synthwave / outrun, on peut faire respirer le son

/// au tempo avec un LFO synchrone au BPM.

///
---------------------------------------------------------------------
-------

fn pump(x: Tensor<B, 1>, sample_rate: usize, bpm: f32, depth: f32)
-> Tensor<B, 1> {

    let n = x.dims()[0];

    let device = x.device();


    let beats_hz = bpm / 60.0;

    let t = time_axis(n, sample_rate, &device);
```

```
    // LFO 0..1
    let lfo = t
        .mul_scalar(2.0 * PI * beats_hz)
        .sin()
        .add_scalar(1.0)
        .div_scalar(2.0);

    // Courbe un peu plus marquée: on "carré" le LFO.
    let duck = lfo.clone() * lfo;

    // Gain entre (1-depth) et 1
    let gain = Tensor::<B, 1>::ones([n], &device) -
duck.mul_scalar(depth);

    x * gain
}

///
---------------------------------------------------------------------
-------
/// EXEMPLE FINAL — CHAÎNE “SYNTHWAVE”
///
---------------------------------------------------------------------
-------
/// Ce n’est pas de la magie: on ne transforme pas n’importe quel
son
```

```
/// en Juno-106 tout seul.

/// En revanche, cette chaîne redonne:

/// - de la couleur (saturation),

/// - du relief (chorus),

/// - de l’espace (delay),

/// - du mouvement (pump),

/// - un équilibre spectral plus rétro/futuriste.

///
---------------------------------------------------------------------
-------

fn synthwaveify(x: Tensor<B, 1>, sample_rate: usize) -> Tensor<B, 1>
{

    // 1) Plus de caractère dès l’entrée.

    let stage1 = normalize_peak(soft_clip_tanh(gain(x, 1.8), 1.0),
0.95);



    // 2) On sépare un "corps" plus doux et une "brillance" plus
nerveuse.

    let body = low_pass_moving_average(stage1.clone(), 6);

    let shimmer = high_pass_from_lowpass(stage1.clone(),
32).mul_scalar(0.35);



    let tone = normalize_peak(body.mul_scalar(0.90) + shimmer,
0.95);



    // 3) On épaissit.

    let widened = chorus(tone, sample_rate);
```

```
    // 4) Echo rétro.
    let echoed = multi_tap_delay(widened, sample_rate, 220, 4, 0.55,
0.28);


    // 5) Léger sidechain/pump.
    let pumped = pump(echoed, sample_rate, 100.0, 0.25);


    // 6) Finition.
    normalize_peak(soft_clip_tanh(pumped, 1.2), 0.95)
}


fn main() -> Result<(), Box<dyn Error>> {
    let device = Device::default();
    let sample_rate = 44_100usize;


    //
------------------------------------------------------------------
---
    // Démo 1: on fabrique une source de test synthétique.
    //
------------------------------------------------------------------
---
    let source = demo_source(sample_rate, &device);
    write_wav_mono_f32(
        "00_source.wav",
```

```
        &tensor_to_samples(source.clone()),
        sample_rate as u32,
    )?;

    // 1) Gain
    let ex1 = normalize_peak(gain(source.clone(), 0.5), 0.95);
    write_wav_mono_f32("01_gain.wav", &tensor_to_samples(ex1),
sample_rate as u32)?;

    // 2) Hard clip
    let ex2 = normalize_peak(hard_clip(source.clone(), 0.40), 0.95);
    write_wav_mono_f32("02_hard_clip.wav", &tensor_to_samples(ex2),
sample_rate as u32)?;

    // 3) Soft clip
    let ex3 = normalize_peak(soft_clip_tanh(source.clone(), 2.2),
0.95);
    write_wav_mono_f32("03_soft_clip.wav", &tensor_to_samples(ex3),
sample_rate as u32)?;

    // 4) Tremolo
    let ex4 = normalize_peak(tremolo(source.clone(), sample_rate,
4.0, 0.70), 0.95);
    write_wav_mono_f32("04_tremolo.wav", &tensor_to_samples(ex4),
sample_rate as u32)?;

    // 5) Ring mod
```

```
    let ex5 = normalize_peak(ring_mod(source.clone(), sample_rate,
35.0), 0.95);

    write_wav_mono_f32("05_ring_mod.wav", &tensor_to_samples(ex5),
sample_rate as u32)?;



    // 6) Low-pass

    let ex6 = normalize_peak(low_pass_moving_average(source.clone(),
10), 0.95);

    write_wav_mono_f32("06_low_pass.wav", &tensor_to_samples(ex6),
sample_rate as u32)?;



    // 7) High-pass

    let ex7 = normalize_peak(high_pass_from_lowpass(source.clone(),
20), 0.95);

    write_wav_mono_f32("07_high_pass.wav", &tensor_to_samples(ex7),
sample_rate as u32)?;



    // 8) Chorus

    let ex8 = chorus(source.clone(), sample_rate);

    write_wav_mono_f32("08_chorus.wav", &tensor_to_samples(ex8),
sample_rate as u32)?;



    // 9) Delay

    let ex9 = multi_tap_delay(source.clone(), sample_rate, 210, 4,
0.55, 0.35);

    write_wav_mono_f32("09_delay.wav", &tensor_to_samples(ex9),
sample_rate as u32)?;
```

```
    // 10) Chaîne complète "synthwave"

    let ex10 = synthwaveify(source.clone(), sample_rate);

    write_wav_mono_f32("10_synthwave_demo.wav",
&tensor_to_samples(ex10), sample_rate as u32)?;



    //
------------------------------------------------------------------------
---

    // Démo 2: si un fichier input.wav existe, on le traite aussi.

    //
------------------------------------------------------------------------
---

    if Path::new("input.wav").exists() {

        let (samples, spec) = read_wav_mono_f32("input.wav")?;

        let input = tensor_from_samples(&samples, &device);


        let output = synthwaveify(input, spec.sample_rate as usize);


        write_wav_mono_f32(

            "99_input_synthwave.wav",

            &tensor_to_samples(output),

            spec.sample_rate,

        )?;


        println!("Fichier traité: 99_input_synthwave.wav");

    } else {
```

```
        println!("Pas de input.wav trouvé, uniquement les démos
synthétiques ont été générées.");

    }



    Ok(())

}
```

Ce que tu obtiens, dans l'ordre

00_source.wav est la source propre.
Puis tu entends successivement gain, hard clip, soft clip, tremolo, ring mod, low-pass, high-pass, chorus, delay, puis 10_synthwave_demo.wav qui combine tout en chaîne.

Pour coller encore plus au brief, l'étape suivante serait une vraie version **stéréo** et une variante **Conv1d** pour écrire les filtres FIR comme des kernels 1D ; Burn a bien un module Conv1d qui prend des tenseurs de forme [batch, channels, length]. (Docs.rs)

Pour le GPU, tu peux remplacer l'alias backend CPU par burn_wgpu::Wgpu et le device par WgpuDevice::DefaultDevice. Le backend WGPU documenté par Burn supporte Vulkan, OpenGL, DX12, Metal et WebGPU, et l'initialisation manuelle n'est utile que si tu veux choisir explicitement l'API ou des options runtime. (Docs.rs)

Je peux te faire juste après une **version stéréo complète** avec un **chorus gauche/droite** et un **delay ping-pong**.

rn-wgpu s'appuie sur un CubeBackend<WgpuRuntime, ...> pour exécuter les tenseurs sur GPU. (Docs.rs)

Je te donne une version cohérente avec Burn 0.20.1 ; selon ton setup exact, il peut y avoir 1 ou 2 imports à ajuster.

Cargo.toml

```
[package]
name = "burn-audio-playground"
version = "0.1.0"
edition = "2021"
```

```
[dependencies]
burn = "0.20.1"
burn-ndarray = "0.20.1"
hound = "3.5.1"
```

src/main.rs

```
use burn::{
    prelude::s,
    tensor::{Int, Tensor, TensorData},
};
use burn_ndarray::{NdArray, NdArrayDevice};
use std::{error::Error, f32::consts::PI, path::Path};

/// ---------------------------------------------------------------------------
/// BACKEND
/// ---------------------------------------------------------------------------
/// Version simple: CPU avec ndarray.
///
/// Pour passer sur GPU plus tard, remplace simplement ces lignes par:
///
/// use burn_wgpu::{Wgpu, WgpuDevice};
/// type B = Wgpu;
/// type Device = WgpuDevice;
///
/// et dans main():
/// let device = WgpuDevice::DefaultDevice;
///
/// Le reste du code DSP ne change pas.
/// ---------------------------------------------------------------------------
type B = NdArray<f32>;
type Device = NdArrayDevice;

/// ---------------------------------------------------------------------------
/// I/O WAV (mono 16-bit pour garder le code court et clair)
/// ---------------------------------------------------------------------------

/// Lit un WAV mono 16-bit et renvoie les samples normalisés en [-1.0, 1.0].
fn read_wav_mono_f32(path: &str) -> Result<(Vec<f32>, hound::WavSpec), Box<dyn
Error>> {
    let mut reader = hound::WavReader::open(path)?;
    let spec = reader.spec();

    if spec.channels != 1 {
        return Err("Ce code de démo attend un WAV mono.".into());
    }
```

```
    if spec.bits_per_sample != 16 || spec.sample_format != hound::SampleFormat::Int {
        return Err("Ce code de démo attend un WAV 16-bit PCM.".into());
    }

    let samples: Vec<f32> = reader
        .samples::<i16>()
        .map(|s| s.map(|v| v as f32 / i16::MAX as f32))
        .collect::<Result<Vec<_>, _>>()?;

    Ok((samples, spec))
}

/// Écrit un WAV mono 16-bit à partir de samples en [-1.0, 1.0].
fn write_wav_mono_f32(path: &str, samples: &[f32], sample_rate: u32) -> Result<(),
Box<dyn Error>> {
    let spec = hound::WavSpec {
        channels: 1,
        sample_rate,
        bits_per_sample: 16,
        sample_format: hound::SampleFormat::Int,
    };

    let mut writer = hound::WavWriter::create(path, spec)?;
    for &x in samples {
        let y = x.clamp(-1.0, 1.0);
        writer.write_sample((y * i16::MAX as f32) as i16)?;
    }
    writer.finalize()?;
    Ok(())
}

/// Convertit un Vec<f32> vers un Tensor 1D Burn.
fn tensor_from_samples(samples: &[f32], device: &Device) -> Tensor<B, 1> {
    Tensor::<B, 1>::from_data(TensorData::new(samples.to_vec(), [samples.len()]), device)
}

/// Convertit un Tensor Burn vers Vec<f32>.
fn tensor_to_samples(x: Tensor<B, 1>) -> Vec<f32> {
    x.into_data().to_vec::<f32>().unwrap()
}

/// ---------------------------------------------------------------------------
/// OUTILS DE BASE
/// ---------------------------------------------------------------------------

/// Crée un axe temporel t = [0, 1/sr, 2/sr, ...].
fn time_axis(n: usize, sample_rate: usize, device: &Device) -> Tensor<B, 1> {
    Tensor::<B, 1, Int>::arange(0..n as i64, device)
```

```
        .float()
        .div_scalar(sample_rate as f32)
}

/// Génère une onde sinusoïdale pure.
fn sine_wave(freq_hz: f32, seconds: f32, sample_rate: usize, device: &Device) -> Tensor<B,
1> {
    let n = (seconds * sample_rate as f32) as usize;
    let t = time_axis(n, sample_rate, device);

    // sin(2πft)
    t.mul_scalar(2.0 * PI * freq_hz).sin()
}

/// Normalise un signal pour que son pic soit égal à `target_peak`.
fn normalize_peak(x: Tensor<B, 1>, target_peak: f32) -> Tensor<B, 1> {
    let peak = x.clone().abs().max().into_scalar();

    // Évite une division par zéro sur un signal silencieux.
    if peak < 1e-8 {
        x
    } else {
        x.div_scalar(peak).mul_scalar(target_peak)
    }
}

/// Mélange dry/wet.
/// wet_amount = 0.0 => 100% dry
/// wet_amount = 1.0 => 100% wet
fn mix(dry: Tensor<B, 1>, wet: Tensor<B, 1>, wet_amount: f32) -> Tensor<B, 1> {
    dry.clone().mul_scalar(1.0 - wet_amount) + wet.mul_scalar(wet_amount)
}

/// Crée un signal de test plus intéressant qu'une simple sinusoïde:
/// un mélange de grave / médium / aigu.
fn demo_source(sample_rate: usize, device: &Device) -> Tensor<B, 1> {
    let bass = sine_wave(110.0, 4.0, sample_rate, device).mul_scalar(0.55);
    let mid = sine_wave(220.0, 4.0, sample_rate, device).mul_scalar(0.30);
    let high = sine_wave(880.0, 4.0, sample_rate, device).mul_scalar(0.15);

    normalize_peak(bass + mid + high, 0.90)
}

/// Décale le signal vers la droite avec remplissage par zéros.
/// Utile pour construire delays et FIR "maison".
fn shift_right_zero(x: Tensor<B, 1>, shift: usize) -> Tensor<B, 1> {
    let n = x.dims()[0];
    let device = x.device();
```

```
    if shift == 0 {
        return x;
    }
    if shift >= n {
        return Tensor::<B, 1>::zeros([n], &device);
    }

    let core = x.slice(s![..(n - shift)]);
    let mut out = Tensor::<B, 1>::zeros([n], &device);
    out = out.slice_assign(s![(shift)..], core);
    out
}

/// Décale le signal vers la gauche avec remplissage par zéros.
fn shift_left_zero(x: Tensor<B, 1>, shift: usize) -> Tensor<B, 1> {
    let n = x.dims()[0];
    let device = x.device();

    if shift == 0 {
        return x;
    }
    if shift >= n {
        return Tensor::<B, 1>::zeros([n], &device);
    }

    let core = x.slice(s![(shift)..]);
    let mut out = Tensor::<B, 1>::zeros([n], &device);
    out = out.slice_assign(s![..(n - shift)], core);
    out
}

/// ---------------------------------------------------------------------------
/// EXEMPLE 1 — GAIN (le plus simple)
/// ---------------------------------------------------------------------------
/// Multiplie chaque sample par un coefficient.
/// ---------------------------------------------------------------------------
fn gain(x: Tensor<B, 1>, amount: f32) -> Tensor<B, 1> {
    x.mul_scalar(amount)
}

/// ---------------------------------------------------------------------------
/// EXEMPLE 2 — HARD CLIP
/// ---------------------------------------------------------------------------
/// Coupe brutalement les crêtes au-delà d'un seuil.
/// Son plus agressif, plus "sale".
/// ---------------------------------------------------------------------------
fn hard_clip(x: Tensor<B, 1>, threshold: f32) -> Tensor<B, 1> {
```

```
    x.clamp_min(-threshold)
        .clamp_max(threshold)
        .div_scalar(threshold)
}

/// ---------------------------------------------------------------------------
/// EXEMPLE 3 — SOFT CLIP / SATURATION
/// ---------------------------------------------------------------------------
/// tanh() donne une saturation plus douce que le hard-clip.
/// Très pratique pour “redonner du caractère”.
/// ---------------------------------------------------------------------------
fn soft_clip_tanh(x: Tensor<B, 1>, drive: f32) -> Tensor<B, 1> {
    x.mul_scalar(drive).tanh()
}

/// ---------------------------------------------------------------------------
/// EXEMPLE 4 — TREMOLO
/// ---------------------------------------------------------------------------
/// Modulation d’amplitude par un LFO lent.
/// C’est un grand classique rétro.
/// ---------------------------------------------------------------------------
fn tremolo(x: Tensor<B, 1>, sample_rate: usize, lfo_hz: f32, depth: f32) -> Tensor<B, 1> {
    let n = x.dims()[0];
    let device = x.device();

    let t = time_axis(n, sample_rate, &device);

    // LFO entre 0 et 1.
    let lfo = t
        .mul_scalar(2.0 * PI * lfo_hz)
        .sin()
        .add_scalar(1.0)
        .div_scalar(2.0);

    // depth = 0 => aucun effet
    // depth = 1 => amplitude suit totalement le LFO
    let amp = lfo.mul_scalar(depth).add_scalar(1.0 - depth);

    x * amp
}

/// ---------------------------------------------------------------------------
/// EXEMPLE 5 — RING MODULATION
/// ---------------------------------------------------------------------------
/// On multiplie le signal par une autre sinusoïde.
/// Ça crée des sidebands métalliques / futuristes.
/// ---------------------------------------------------------------------------
fn ring_mod(x: Tensor<B, 1>, sample_rate: usize, mod_hz: f32) -> Tensor<B, 1> {
```

```
    let n = x.dims()[0];
    let device = x.device();

    let t = time_axis(n, sample_rate, &device);
    let carrier = t.mul_scalar(2.0 * PI * mod_hz).sin();

    x * carrier
}

/// ---------------------------------------------------------------------------
/// EXEMPLE 6 — LOW-PASS “MAISON”
/// ---------------------------------------------------------------------------
/// Ici on fait un FIR très simple: moyenne glissante symétrique.
/// Plus `radius` est grand, plus on lisse les aigus.
/// ---------------------------------------------------------------------------
fn low_pass_moving_average(x: Tensor<B, 1>, radius: usize) -> Tensor<B, 1> {
    let n = x.dims()[0];
    let device = x.device();

    let mut acc = Tensor::<B, 1>::zeros([n], &device);
    let mut taps = 0usize;

    // Centre
    acc = acc + x.clone();
    taps += 1;

    // Voisins gauche / droite
    for k in 1..=radius {
        acc = acc + shift_right_zero(x.clone(), k);
        acc = acc + shift_left_zero(x.clone(), k);
        taps += 2;
    }

    acc.div_scalar(taps as f32)
}

/// ---------------------------------------------------------------------------
/// EXEMPLE 7 — HIGH-PASS par soustraction
/// ---------------------------------------------------------------------------
/// On enlève la version low-pass au signal original.
/// Ce qui reste = surtout les variations rapides => le haut du spectre.
/// ---------------------------------------------------------------------------
fn high_pass_from_lowpass(x: Tensor<B, 1>, radius: usize) -> Tensor<B, 1> {
    let low = low_pass_moving_average(x.clone(), radius);
    x - low
}

/// ---------------------------------------------------------------------------
```

```
/// EXEMPLE 8 — CHORUS TRÈS SIMPLE
/// ------------------------------------------------------------------------
/// Un vrai chorus module le délai dans le temps.
/// Ici, on fait une version simple avec deux micro-delays fixes.
/// Ça épaissit déjà bien le son.
/// ------------------------------------------------------------------------
fn chorus(x: Tensor<B, 1>, sample_rate: usize) -> Tensor<B, 1> {
    let d1 = (0.015 * sample_rate as f32) as usize; // 15 ms
    let d2 = (0.023 * sample_rate as f32) as usize; // 23 ms

    let wet =
        shift_right_zero(x.clone(), d1).mul_scalar(0.35) +
        shift_right_zero(x.clone(), d2).mul_scalar(0.25);

    normalize_peak(mix(x, wet, 0.40), 0.95)
}

/// ------------------------------------------------------------------------
/// EXEMPLE 9 — MULTI-TAP DELAY
/// ------------------------------------------------------------------------
/// Pas de feedback récursif ici: on fabrique plusieurs copies retardées,
/// de plus en plus faibles. Très simple, mais super utile.
/// ------------------------------------------------------------------------
fn multi_tap_delay(
    x: Tensor<B, 1>,
    sample_rate: usize,
    delay_ms: usize,
    taps: usize,
    decay: f32,
    wet_amount: f32,
) -> Tensor<B, 1> {
    let n = x.dims()[0];
    let device = x.device();
    let base_delay = sample_rate * delay_ms / 1000;

    let mut wet = Tensor::<B, 1>::zeros([n], &device);

    for tap in 1..=taps {
        let shift = base_delay * tap;
        let gain = decay.powi(tap as i32);

        wet = wet + shift_right_zero(x.clone(), shift).mul_scalar(gain);
    }

    normalize_peak(mix(x, wet, wet_amount), 0.95)
}

/// ------------------------------------------------------------------------
```

```
/// EXEMPLE 10 — “PUMP” type sidechain
/// ---------------------------------------------------------------------------
/// Pour le côté synthwave / outrun, on peut faire respirer le son
/// au tempo avec un LFO synchrone au BPM.
/// ---------------------------------------------------------------------------
fn pump(x: Tensor<B, 1>, sample_rate: usize, bpm: f32, depth: f32) -> Tensor<B, 1> {
    let n = x.dims()[0];
    let device = x.device();

    let beats_hz = bpm / 60.0;
    let t = time_axis(n, sample_rate, &device);

    // LFO 0..1
    let lfo = t
        .mul_scalar(2.0 * PI * beats_hz)
        .sin()
        .add_scalar(1.0)
        .div_scalar(2.0);

    // Courbe un peu plus marquée: on "carré" le LFO.
    let duck = lfo.clone() * lfo;

    // Gain entre (1-depth) et 1
    let gain = Tensor::<B, 1>::ones([n], &device) - duck.mul_scalar(depth);

    x * gain
}

/// ---------------------------------------------------------------------------
/// EXEMPLE FINAL — CHAÎNE “SYNTHWAVE”
/// ---------------------------------------------------------------------------
/// Ce n’est pas de la magie: on ne transforme pas n’importe quel son
/// en Juno-106 tout seul.
/// En revanche, cette chaîne redonne:
/// - de la couleur (saturation),
/// - du relief (chorus),
/// - de l’espace (delay),
/// - du mouvement (pump),
/// - un équilibre spectral plus rétro/futuriste.
/// ---------------------------------------------------------------------------
fn synthwaveify(x: Tensor<B, 1>, sample_rate: usize) -> Tensor<B, 1> {
    // 1) Plus de caractère dès l’entrée.
    let stage1 = normalize_peak(soft_clip_tanh(gain(x, 1.8), 1.0), 0.95);

    // 2) On sépare un "corps" plus doux et une "brillance" plus nerveuse.
    let body = low_pass_moving_average(stage1.clone(), 6);
    let shimmer = high_pass_from_lowpass(stage1.clone(), 32).mul_scalar(0.35);
```

```
    let tone = normalize_peak(body.mul_scalar(0.90) + shimmer, 0.95);

    // 3) On épaissit.
    let widened = chorus(tone, sample_rate);

    // 4) Echo rétro.
    let echoed = multi_tap_delay(widened, sample_rate, 220, 4, 0.55, 0.28);

    // 5) Léger sidechain/pump.
    let pumped = pump(echoed, sample_rate, 100.0, 0.25);

    // 6) Finition.
    normalize_peak(soft_clip_tanh(pumped, 1.2), 0.95)
}

fn main() -> Result<(), Box<dyn Error>> {
    let device = Device::default();
    let sample_rate = 44_100usize;

    // ----------------------------------------------------------------------
    // Démo 1: on fabrique une source de test synthétique.
    // ----------------------------------------------------------------------
    let source = demo_source(sample_rate, &device);
    write_wav_mono_f32(
        "00_source.wav",
        &tensor_to_samples(source.clone()),
        sample_rate as u32,
    )?;

    // 1) Gain
    let ex1 = normalize_peak(gain(source.clone(), 0.5), 0.95);
    write_wav_mono_f32("01_gain.wav", &tensor_to_samples(ex1), sample_rate as u32)?;

    // 2) Hard clip
    let ex2 = normalize_peak(hard_clip(source.clone(), 0.40), 0.95);
    write_wav_mono_f32("02_hard_clip.wav", &tensor_to_samples(ex2), sample_rate as
u32)?;

    // 3) Soft clip
    let ex3 = normalize_peak(soft_clip_tanh(source.clone(), 2.2), 0.95);
    write_wav_mono_f32("03_soft_clip.wav", &tensor_to_samples(ex3), sample_rate as
u32)?;

    // 4) Tremolo
    let ex4 = normalize_peak(tremolo(source.clone(), sample_rate, 4.0, 0.70), 0.95);
    write_wav_mono_f32("04_tremolo.wav", &tensor_to_samples(ex4), sample_rate as
u32)?;
```

```
    // 5) Ring mod
    let ex5 = normalize_peak(ring_mod(source.clone(), sample_rate, 35.0), 0.95);
    write_wav_mono_f32("05_ring_mod.wav", &tensor_to_samples(ex5), sample_rate as
u32)?;

    // 6) Low-pass
    let ex6 = normalize_peak(low_pass_moving_average(source.clone(), 10), 0.95);
    write_wav_mono_f32("06_low_pass.wav", &tensor_to_samples(ex6), sample_rate as
u32)?;

    // 7) High-pass
    let ex7 = normalize_peak(high_pass_from_lowpass(source.clone(), 20), 0.95);
    write_wav_mono_f32("07_high_pass.wav", &tensor_to_samples(ex7), sample_rate as
u32)?;

    // 8) Chorus
    let ex8 = chorus(source.clone(), sample_rate);
    write_wav_mono_f32("08_chorus.wav", &tensor_to_samples(ex8), sample_rate as u32)?;

    // 9) Delay
    let ex9 = multi_tap_delay(source.clone(), sample_rate, 210, 4, 0.55, 0.35);
    write_wav_mono_f32("09_delay.wav", &tensor_to_samples(ex9), sample_rate as u32)?;

    // 10) Chaîne complète "synthwave"
    let ex10 = synthwaveify(source.clone(), sample_rate);
    write_wav_mono_f32("10_synthwave_demo.wav", &tensor_to_samples(ex10),
sample_rate as u32)?;

    // ----------------------------------------------------------------------
    // Démo 2: si un fichier input.wav existe, on le traite aussi.
    // ----------------------------------------------------------------------
    if Path::new("input.wav").exists() {
        let (samples, spec) = read_wav_mono_f32("input.wav")?;
        let input = tensor_from_samples(&samples, &device);

        let output = synthwaveify(input, spec.sample_rate as usize);

        write_wav_mono_f32(
            "99_input_synthwave.wav",
            &tensor_to_samples(output),
            spec.sample_rate,
        )?;

        println!("Fichier traité: 99_input_synthwave.wav");
    } else {
        println!("Pas de input.wav trouvé, uniquement les démos synthétiques ont été
générées.");
    }
```

```
    Ok(())
}
```

Ce que tu obtiens, dans l’ordre

`00_source.wav` est la source propre.
Puis tu entends successivement `gain`, `hard clip`, `soft clip`, `tremolo`, `ring mod`, `low-pass`, `high-pass`, `chorus`, `delay`, puis `10_synthwave_demo.wav` qui combine tout en chaîne.

Pour coller encore plus au brief, l’étape suivante serait une vraie version **stéréo** et une variante **Conv1d** pour écrire les filtres FIR comme des kernels 1D ; Burn a bien un module `Conv1d` qui prend des tenseurs de forme `[batch, channels, length]`. (Docs.rs)

Pour le GPU, tu peux remplacer l’alias backend CPU par `burn_wgpu::Wgpu` et le device par `WgpuDevice::DefaultDevice`. Le backend WGPU documenté par Burn supporte Vulkan, OpenGL, DX12, Metal et WebGPU, et l’initialisation manuelle n’est utile que si tu veux choisir explicitement l’API ou des options runtime. (Docs.rs)

Je peux te faire juste après une **version stéréo complète** avec un **chorus gauche/droite** et un **delay ping-pong**.